

Section 96 Editorial — Master Continuous Geometry Mixed Contest

A. Safe Region Diameter

How to find it

First recover the convex intersection polygon with the half-plane deque. Then use rotating calipers to find its farthest pair of vertices.

Correctness

The deque produces exactly the feasible polygon. A diameter of a convex polygon has antipodal endpoints; rotating calipers enumerates every antipodal pair while the opposite pointer moves only forward, so the largest tested distance is the region diameter.

Complexity

$O(n \log n)$ time for sorting half-planes plus $O(h)$ calipers time, and $O(n)$ memory.

C++

```
#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        }
    };
}
P operator-(P b)const {
    return {
        x-b.x,y-b.y
    }
}
P operator*(double k)const {
    return {
        x*k,y*k
    }
}
}
;
double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
```

```

double norm(P a) {
    return hypot(a.x,a.y);
}

struct L {
    P p,v;
    double angle;
}

;
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}

vector<P>halfplanes(vector<pair<P,P>>raw) {
    vector<L>a;
    for(auto [p,q]:raw) {
        P v=q-p;
        a.push_back({p,v,atan2(v.y,v.x)});
    }
    sort(a.begin(),a.end(),[](L x,L y){return x.angle<y.angle;});
    vector<L>u;
    for(L z:a) {
        if(!u.empty()&&abs(cross(u.back().v,z.v))<=EPS&&dot(u.back().v,z.v)>0) {
            if(cross(u.back().v,z.p-u.back().p)>EPS)u.back().v=z;
        }
        else u.push_back(z);
    }
    deque<L>q;
    deque<P>pts;
    auto inside=[](L l,P p) {
        return cross(l.v,p-l.p)>=-EPS;
    }
    ;
    for(L z:u) {
        while(!pts.empty()&&!inside(z,pts.back()))pts.pop_back(),q.pop_back();
        while(!pts.empty()&&!inside(z,pts.front()))pts.pop_front(),q.pop_front();
        if(!q.empty()&&abs(cross(q.back().v,z.v))<=EPS)return {
        }
        ;
        if(!q.empty())pts.push_back(meet(q.back(),z));
        q.push_back(z);
    }
    while(!pts.empty()&&!inside(q.front(),pts.back()))pts.pop_back(),q.pop_back();
    while(!pts.empty()&&!inside(q.back(),pts.front()))pts.pop_front(),q.pop_front();
    if(q.size()<3)return {
    }
    ;
    pts.push_back(meet(q.back(),q.front()));
    return vector<P>(pts.begin(),pts.end());
}

double diameter(vector<P> p) {
    int n=p.size();
    if(n<2)return 0;
    if(n==2)return norm(p[0]-p[1]);
    int opposite=1;
    double answer=0;
    for(int i=0;i<n;i++) {
        int next=(i+1)%n;
        while(abs(cross(p[next]-p[i],p[(opposite+1)%n]-p[i]))
            >abs(cross(p[next]-p[i],p[opposite]-p[i]))+EPS)
            opposite=(opposite+1)%n;
        answer=max(answer,norm(p[i]-p[opposite]));
        answer=max(answer,norm(p[next]-p[opposite]));
    }
}

```

```

    return answer;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    int n;
    cin>>n;
    vector<pair<P,P>>l(n);
    for(auto&[a,b]:l)cin>>a.x>>a.y>>b.x>>b.y;
    cout<<diameter(halplanes(l))<<'\n';
}

```

Python

```

import sys, math, random
from collections import deque
EPS = 1e-10

def add(a, b):
    return (a[0] + b[0], a[1] + b[1])

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def mul(a, k):
    return (a[0] * k, a[1] * k)

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def intersection(a, b):
    t = cross(sub(b[0], a[0]), b[1]) / cross(a[1], b[1])
    return add(a[0], mul(a[1], t))

def halplanes(raw):
    lines = []
    for a, b in raw:
        v = sub(b, a)
        lines.append((a, v, math.atan2(v[1], v[0])))
    lines.sort(key=lambda z: z[2])
    unique = []
    for p, v, angle in lines:
        if unique and abs(cross(unique[-1][1], v)) <= EPS and (dot(unique[-1][1], v) > 0):
            if cross(unique[-1][1], sub(p, unique[-1][0])) > EPS:
                unique[-1] = (p, v, angle)
            else:
                unique.append((p, v, angle))
    dq = deque()
    points = deque()

    def inside(line, p):
        return cross(line[1], sub(p, line[0])) >= -EPS
    for line in unique:
        while points and (not inside(line, points[-1])):
            points.pop()
            dq.pop()
        while points and (not inside(line, points[0])):
            points.popleft()

```

```

        dq.popleft()
    if dq and abs(cross(dq[-1][1], line[1])) <= EPS:
        return []
    if dq:
        points.append(intersection(dq[-1], line))
    dq.append(line)
    while points and (not inside(dq[0], points[-1])):
        points.pop()
        dq.pop()
    while points and (not inside(dq[-1], points[0])):
        points.popleft()
        dq.popleft()
    if len(dq) < 3:
        return []
    points.append(intersection(dq[-1], dq[0]))
    return list(points)

def diameter(poly):
    n = len(poly)
    if n < 2:
        return 0.0
    if n == 2:
        return math.dist(poly[0], poly[1])
    opposite = 1
    answer = 0.0
    for i in range(n):
        nxt = (i + 1) % n
        edge = sub(poly[nxt], poly[i])
        while abs(cross(edge, sub(poly[(opposite + 1) % n], poly[i]))) > abs(cross(edge, sub(poly[opposite], poly[i]))) - EPS:
            opposite = (opposite + 1) % n
        answer = max(answer, math.dist(poly[i], poly[opposite]))
        answer = max(answer, math.dist(poly[nxt], poly[opposite]))
    return answer

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n = int(d[0])
    raw = []
    for i in range(n):
        x1, y1, x2, y2 = d[1 + 4 * i:5 + 4 * i]
        raw.append((x1, y1), (x2, y2))
    print(f'{diameter(halflanes(raw)):.10f}')
if __name__ == '__main__':
    main()

```

B. Round Table Clearance

How to find it

Offset every counterclockwise edge inward by radius r ; exploit the convex polygon edge order for a linear deque feasibility test, then binary-search r .

Correctness

A center supports radius r exactly when its distance from every edge is at least r . The deque finds a candidate intersection in edge order, and the final all-edge witness check certifies that candidate before declaring feasibility.

Complexity

$O(n \log \text{precision})$ time and $O(n)$ memory.

C++

```
#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        };
    }
    P operator-(P b)const {
        return {
            x-b.x,y-b.y
        };
    }
    P operator*(double k)const {
        return {
            x*k,y*k
        };
    }
};
double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
}
struct L {
    P p,v;
    double angle;
};
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
vector<P>halfplanes(vector<pair<P,P>>raw, bool presorted=false) {
    vector<L>a;
    for(auto[p,q]:raw) {
        P v=q-p;
        a.push_back({p,v,atan2(v.y,v.x)});
    }
    if(!presorted)sort(a.begin(),a.end(),[](L x,L y){return x.angle<y.angle;});
    vector<L>u;
    for(L z:a) {
        if(!u.empty()&&abs(cross(u.back().v,z.v))<=EPS&&dot(u.back().v,z.v)>0) {
            if(cross(u.back().v,z.p-u.back().p)>EPS)u.back().v=z;
        }
        else u.push_back(z);
    }
    deque<L>q;
    deque<P>pts;
    auto inside=[](L l,P p) {
        return cross(l.v,p-l.p)>=-EPS;
    };
}
```

```

}
;
for(L z:u) {
    while(!pts.empty() && !inside(z, pts.back())) pts.pop_back(), q.pop_back();
    while(!pts.empty() && !inside(z, pts.front())) pts.pop_front(), q.pop_front();
    if(!q.empty() && abs(cross(q.back().v, z.v)) <= EPS) return {
    }
    ;
    if(!q.empty()) pts.push_back(meet(q.back(), z));
    q.push_back(z);
}
while(!pts.empty() && !inside(q.front(), pts.back())) pts.pop_back(), q.pop_back();
while(!pts.empty() && !inside(q.back(), pts.front())) pts.pop_front(), q.pop_front();
if(q.size() < 3) return {
}
;
pts.push_back(meet(q.back(), q.front()));
return vector<P>(pts.begin(), pts.end());
}

double area(vector<P>p) {
    double s=0;
    for(int i=0; i<(int)p.size(); i++) s+=cross(p[i], p[(i+1)%p.size()]);
    return abs(s)/2;
}

vector<P>shifted(vector<P>p, double r) {
    vector<pair<P, P>>l;
    for(int i=0; i<(int)p.size(); i++) {
        P q=p[(i+1)%p.size()], v=q-p[i], s= {
            -v.y*r/norm(v), v.x*r/norm(v)
        }
        ;
        l.push_back({p[i]+s, q+s});
    }
    vector<P>result=halfplanes(l, true);
    if(result.empty()) return result;
    P witness=result[0];
    for(auto [a, b]:l) if(cross(b-a, witness-a) < -EPS) return {
    }
    ;
    return result;
}

double inradius(vector<P>p) {
    int at=0;
    for(int i=1; i<(int)p.size(); i++) {
        P vi=p[(i+1)%p.size()]-p[i], va=p[(at+1)%p.size()]-p[at];
        if(atan2(vi.y, vi.x) < atan2(va.y, va.x)) at=i;
    }
    rotate(p.begin(), p.begin()+at, p.end());
    double minx=p[0].x, maxx=p[0].x, miny=p[0].y, maxy=p[0].y;
    for(P x:p) minx=min(minx, x.x), maxx=max(maxx, x.x), miny=min(miny, x.y), maxy=max(maxy, x.y);
    double lo=0, hi=max(maxx-minx, maxy-miny)+1;
    for(int z=0; z<70; z++) {
        double m=(lo+hi)/2;
        (shifted(p, m).empty()?hi:lo)=m;
    }
    return lo;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    int n;
    cin>>n;

```

```

vector<P>p(n);
for(auto&x:p)cin>>x.x>>x.y;
cout<<inradius(p)<<'\n';
}

```

Python

```

import sys, math, random
from collections import deque
EPS = 1e-10

def add(a, b):
    return (a[0] + b[0], a[1] + b[1])

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def mul(a, k):
    return (a[0] * k, a[1] * k)

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def norm(a):
    return math.hypot(a[0], a[1])

def intersection(a, b):
    t = cross(sub(b[0], a[0]), b[1]) / cross(a[1], b[1])
    return add(a[0], mul(a[1], t))

def halfplanes(raw, presorted=False):
    lines = []
    for a, b in raw:
        v = sub(b, a)
        lines.append((a, v, math.atan2(v[1], v[0])))
    if not presorted:
        lines.sort(key=lambda z: z[2])
    unique = []
    for p, v, angle in lines:
        if unique and abs(cross(unique[-1][1], v)) <= EPS and (dot(unique[-1][1], v) > 0):
            if cross(unique[-1][1], sub(p, unique[-1][0])) > EPS:
                unique[-1] = (p, v, angle)
            else:
                unique.append((p, v, angle))
    dq = deque()
    points = deque()

    def inside(line, p):
        return cross(line[1], sub(p, line[0])) >= -EPS
    for line in unique:
        while points and (not inside(line, points[-1])):
            points.pop()
            dq.pop()
        while points and (not inside(line, points[0])):
            points.popleft()
            dq.popleft()
        if dq and abs(cross(dq[-1][1], line[1])) <= EPS:
            return []
    if dq:

```

```

        points.append(intersection(dq[-1], line))
        dq.append(line)
    while points and (not inside(dq[0], points[-1])):
        points.pop()
        dq.pop()
    while points and (not inside(dq[-1], points[0])):
        points.popleft()
        dq.popleft()
    if len(dq) < 3:
        return []
    points.append(intersection(dq[-1], dq[0]))
    return list(points)

def shifted_polygon(poly, r):
    lines = []
    for i, p in enumerate(poly):
        q = poly[(i + 1) % len(poly)]
        v = sub(q, p)
        length = norm(v)
        shift = (-v[1] * r / length, v[0] * r / length)
        lines.append((add(p, shift), add(q, shift)))
    return halfplanes(lines, True)

def shifted_feasible(edges, r):
    dq = deque()
    points = deque()

    def inside(line, point):
        px, py, vx, vy = line
        return vx * (point[1] - py) - vy * (point[0] - px) >= -EPS

    def meet(a, b):
        ax, ay, avx, avy = a
        bx, by, bvx, bvy = b
        denominator = avx * bvy - avy * bvx
        if abs(denominator) <= EPS:
            return None
        t = ((bx - ax) * bvy - (by - ay) * bvx) / denominator
        return (ax + avx * t, ay + avy * t)

    for px, py, vx, vy, nx, ny in edges:
        line = (px + nx * r, py + ny * r, vx, vy)
        while points and (not inside(line, points[-1])):
            points.pop()
            dq.pop()
        while points and (not inside(line, points[0])):
            points.popleft()
            dq.popleft()
        if dq:
            point = meet(dq[-1], line)
            if point is None:
                return False
            points.append(point)
        dq.append(line)
    while points and (not inside(dq[0], points[-1])):
        points.pop()
        dq.pop()
    while points and (not inside(dq[-1], points[0])):
        points.popleft()
        dq.popleft()
    if len(dq) < 3:
        return False
    witness = meet(dq[-1], dq[0])
    if witness is None:

```



```

        return False
    for px, py, vx, vy, nx, ny in edges:
        if not inside((px + nx * r, py + ny * r, vx, vy), witness):
            return False
    return True

def inradius(poly):
    at = min(range(len(poly)), key=lambda i: math.atan2(poly[(i + 1) % len(poly)][1] - poly[i][1], poly[(i + 1) % len(poly)][0] - poly[i][0]))
    poly = poly[at:] + poly[:at]
    edges = []
    for i, p in enumerate(poly):
        q = poly[(i + 1) % len(poly)]
        vx, vy = (q[0] - p[0], q[1] - p[1])
        length = math.hypot(vx, vy)
        edges.append((p[0], p[1], vx, vy, -vy / length, vx / length))
    low = 0.0
    high = max(max((x for x, y in poly)) - min((x for x, y in poly)), max((y for x, y in poly)) - min((y for x, y in poly)))
    for _ in range(70):
        mid = (low + high) / 2
        if shifted_feasible(edges, mid):
            low = mid
        else:
            high = mid
    return low

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n = int(d[0])
    p = [tuple(d[1 + 2 * i:3 + 2 * i]) for i in range(n)]
    print(f'{inradius(p):.10f}')
if __name__ == '__main__':
    main()

```

C. Sensor Overlap

How to find it

Separate disjoint and containment cases before using the two-sector lens formula.

Correctness

In the proper-overlap case the common region partitions into two sectors minus their center triangles; the other classifications are immediate.

Complexity

$O(1)$ time and memory.

C++

```

#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        }
    }
}

```

```

    };
}
P operator-(P b) const {
    return {
        x-b.x, y-b.y
    };
}
P operator*(double k) const {
    return {
        x*k, y*k
    };
}
}

double cross(P a, P b) {
    return a.x*b.y - a.y*b.x;
}
double dot(P a, P b) {
    return a.x*b.x + a.y*b.y;
}
double norm(P a) {
    return hypot(a.x, a.y);
}
struct L {
    P p, v;
    double angle;
}
;
P meet(L a, L b) {
    double t = cross(b.p - a.p, b.v) / cross(a.v, b.v);
    return a.p + a.v * t;
}
struct C {
    P p;
    double r;
}
;
double overlap(C a, C b) {
    double d = norm(a.p - b.p);
    if (d >= a.r + b.r) return 0;
    if (d <= abs(a.r - b.r)) return PI * min(a.r, b.r) * min(a.r, b.r);
    double x = acos(clamp((d*d + a.r*a.r - b.r*b.r) / (2*d*a.r), -1.0, 1.0)), y = acos(clamp((d*d + b.r*b.r - a.r*a.r) / (2*d*b.r), -1.0, 1.0));
    return a.r*a.r*x + b.r*b.r*y - d*a.r*sin(x);
}
int tangents(C a, C b) {
    double d = norm(a.p - b.p);
    if (d <= EPS) return abs(a.r - b.r) <= EPS ? 1 : 0;
    int ans = 0;
    for (double z : {abs(a.r - b.r), a.r + b.r}) ans += d > z + EPS ? 2 : (abs(d - z) <= EPS);
    return ans;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout << fixed << setprecision(10);
    C a, b;
    cin >> a.p.x >> a.p.y >> a.r >> b.p.x >> b.p.y >> b.r;
    cout << overlap(a, b) << '\n';
}

```

Python

```
import sys, math, random
from collections import deque
EPS = 1e-10

def overlap(c1, c2):
    x1, y1, r1 = c1
    x2, y2, r2 = c2
    d = math.hypot(x1 - x2, y1 - y2)
    if d >= r1 + r2:
        return 0.0
    if d <= abs(r1 - r2):
        return math.pi * min(r1, r2) ** 2
    a = math.acos(max(-1, min(1, (d * d + r1 * r1 - r2 * r2) / (2 * d * r1))))
    b = math.acos(max(-1, min(1, (d * d + r2 * r2 - r1 * r1) / (2 * d * r2))))
    return r1 * r1 * a + r2 * r2 * b - d * r1 * math.sin(a)

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    print(f'{overlap(d[:3], d[3:6]):.10f}')
if __name__ == '__main__':
    main()
```

D. Emergency Broadcast Disk

How to find it

Randomize insertion order and rebuild only when a point violates the current circle, fixing one, two, then three boundary witnesses.

Correctness

At every nesting level the violating fixed points must lie on the new optimum boundary; one to three such points uniquely determine the minimum circle for the processed prefix.

Complexity

Expected $O(n)$ time and $O(n)$ storage.

C++

```
#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        }
    };
}
P operator-(P b)const {
    return {
        x-b.x,y-b.y
    };
};
```

```

    }
    P operator*(double k) const {
        return {
            x*k,y*k
        };
    }
}

double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}

double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}

double norm(P a) {
    return hypot(a.x,a.y);
}

struct L {
    P p,v;
    double angle;
}

P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}

struct C {
    P p;
    double r;
}

double overlap(C a,C b) {
    double d=norm(a.p-b.p);
    if(d>=a.r+b.r) return 0;
    if(d<=abs(a.r-b.r)) return PI*min(a.r,b.r)*min(a.r,b.r);
    double x=acos(clamp((d*d+a.r*a.r-b.r*b.r)/(2*d*a.r),-1.0,1.0)),y=acos(clamp((d*d+b.r*b.r-a.r*a.r)/(2*d*b.r),-1.0,1.0));
    return a.r*a.r*x+b.r*b.r*y-d*a.r*sin(x);
}

int tangents(C a,C b) {
    double d=norm(a.p-b.p);
    if(d<=EPS) return abs(a.r-b.r)<=EPS?-1:0;
    int ans=0;
    for(double z:{abs(a.r-b.r),a.r+b.r}) ans+=d>z+EPS?2:(abs(d-z)<=EPS);
    return ans;
}

C circle2(P a,P b) {
    P c=(a+b)*.5;
    return {
        c,norm(a-b)/2
    };
}

C circle3(P a,P b,P c) {
    double d=2*cross(b-a,c-a);
    double aa=dot(a,a),bb=dot(b,b),cc=dot(c,c);
    P o= {
        (aa*(b.y-c.y)+bb*(c.y-a.y)+cc*(a.y-b.y))/d,(aa*(c.x-b.x)+bb*(a.x-c.x)+cc*(b.x-a.x))/d
    };
    return {
        o,norm(o-a)
    };
}

```

```

    ;
}
C mec(vector<P>p) {
    mt19937 rng(947311);
    shuffle(p.begin(), p.end(), rng);
    C c {
        {
            0,0
        }
        , -1
    }
    ;
    auto outside=[](P x,C c) {
        return c.r<0||norm(x-c.p)>c.r+1e-9;
    }
    ;
    for(int i=0;i<(int)p.size();i++)if(outside(p[i],c)) {
        c= {
            p[i],0
        }
        ;
        for(int j=0;j<i;j++)if(outside(p[j],c)) {
            c=circle2(p[i],p[j]);
            for(int k=0;k<j;k++)if(outside(p[k],c))c=circle3(p[i],p[j],p[k]);
        }
    }
    return c;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    int n;
    cin>>n;
    vector<P>p(n);
    for(auto&x:p)cin>>x.x>>x.y;
    C c=mec(p);
    cout<<c.p.x<<' '<<c.p.y<<' '<<c.r<<'\n';
}

```

Python

```

import sys, math, random
from collections import deque
EPS = 1e-10

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def norm(a):
    return math.hypot(a[0], a[1])

def circle2(a, b):
    c = ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)
    return (c, norm(sub(a, b)) / 2)

```

```

def circle3(a, b, c):
    d = 2 * cross(sub(b, a), sub(c, a))
    ux = (dot(a, a) * (b[1] - c[1]) + dot(b, b) * (c[1] - a[1]) + dot(c, c) * (a[1] - b[1])) / d
    uy = (dot(a, a) * (c[0] - b[0]) + dot(b, b) * (a[0] - c[0]) + dot(c, c) * (b[0] - a[0])) / d
    center = (ux, uy)
    return (center, norm(sub(center, a)))

def minimum_circle(points):
    p = points[:]
    random.Random(947311).shuffle(p)
    circle = ((0.0, 0.0), -1.0)

    def outside(x, c):
        return c[1] < 0 or norm(sub(x, c[0])) > c[1] + 1e-09
    for i in range(len(p)):
        if outside(p[i], circle):
            circle = (p[i], 0.0)
            for j in range(i):
                if outside(p[j], circle):
                    circle = circle2(p[i], p[j])
                    for k in range(j):
                        if outside(p[k], circle):
                            circle = circle3(p[i], p[j], p[k])
    return circle

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n = int(d[0])
    p = [tuple(d[1 + 2 * i:3 + 2 * i]) for i in range(n)]
    c, r = minimum_circle(p)
    print(f'{c[0]:.10f} {c[1]:.10f} {r:.10f}')
if __name__ == '__main__':
    main()

```

E. Delaunay Route

How to find it

Build the Delaunay edges, weight them by Euclidean length, and run Dijkstra from the requested source site.

Correctness

The construction returns exactly the graph permitted by the statement. Euclidean edge lengths are nonnegative, so Dijkstra's settled-distance invariant gives the shortest permitted route.

Complexity

Expected $O(n^2)$ for the local incremental triangulation, followed by $O(n \log n)$ shortest-path work on the linear-size Delaunay graph.

C++

```

#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {

```

```

        x+b.x,y+b.y
    }
    ;
}
P operator-(P b) const {
    return {
        x-b.x,y-b.y
    }
    ;
}
P operator*(double k) const {
    return {
        x*k,y*k
    }
    ;
}
}
;
double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
}
struct L {
    P p,v;
    double angle;
}
;
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
struct C {
    P p;
    double r;
}
;
double overlap(C a,C b) {
    double d=norm(a.p-b.p);
    if(d>=a.r+b.r) return 0;
    if(d<=abs(a.r-b.r)) return PI*min(a.r,b.r)*min(a.r,b.r);
    double x=acos(clamp((d*d+a.r*a.r-b.r*b.r)/(2*d*a.r),-1.0,1.0)),y=acos(clamp((d*d+b.r*b.r-a.r*a.r)/(2*d*b.r),-1.0,1.0));
    return a.r*a.r*x+b.r*b.r*y-d*a.r*sin(x);
}
int tangents(C a,C b) {
    double d=norm(a.p-b.p);
    if(d<=EPS) return abs(a.r-b.r)<=EPS?-1:0;
    int ans=0;
    for(double z:{abs(a.r-b.r),a.r+b.r}) ans+=d>z+EPS?2:(abs(d-z)<=EPS);
    return ans;
}
C circle2(P a,P b) {
    P c=(a+b)*.5;
    return {
        c,norm(a-b)/2
    }
    ;
}
C circle3(P a,P b,P c) {

```

```

double d=2*cross(b-a,c-a);
double aa=dot(a,a),bb=dot(b,b),cc=dot(c,c);
P o= {
    (aa*(b.y-c.y)+bb*(c.y-a.y)+cc*(a.y-b.y))/d,(aa*(c.x-b.x)+bb*(a.x-c.x)+cc*(b.x-a.x))/d
};
;
return {
    o,norm(o-a)
};
;
}
C mec(vector<P>p) {
    mt19937 rng(947311);
    shuffle(p.begin(),p.end(),rng);
    C c {
        {
            0,0
        },
        {-1
    }
    };
    auto outside=[](P x,C c) {
        return c.r<0||norm(x-c.p)>c.r+1e-9;
    };
    ;
    for(int i=0;i<(int)p.size();i++)if(outside(p[i],c)) {
        c= {
            p[i],0
        };
        for(int j=0;j<i;j++)if(outside(p[j],c)) {
            c=circle2(p[i],p[j]);
            for(int k=0;k<j;k++)if(outside(p[k],c))c=circle3(p[i],p[j],p[k]);
        }
    }
    return c;
}
struct T {
    int a,b,c;
};
;
double orient(P a,P b,P c) {
    return cross(b-a,c-a);
}
bool incircle(P a,P b,P c,P p) {
    double ax=a.x-p.x,ay=a.y-p.y,bx=b.x-p.x,by=b.y-p.y,cx=c.x-p.x,cy=c.y-p.y;
    double d=(ax*ax+ay*ay)*(bx*cy-by*cx)-(bx*bx+by*by)*(ax*cy-ay*cx)+(cx*cx+cy*cy)*(ax*by-ay*bx);
    return d>EPS;
}
pair<vector<T>,vector<pair<int,int>>> delaunay(vector<P>p) {
    int n=p.size();
    if(n<3)return {
        {
        },
        {
        }
    };
    ;
    double minx=p[0].x,maxx=p[0].x,miny=p[0].y,maxy=p[0].y;
    for(P x:p)minx=min(minx,x.x),maxx=max(maxx,x.x),miny=min(miny,x.y),maxy=max(maxy,x.y);
    double s=max({maxx-minx,maxy-miny,1.0}),cx=(minx+maxx)/2,cy=(miny+maxy)/2;
    p.push_back({cx-10000*s,cy-8000*s});
    p.push_back({cx+10000*s,cy-8000*s});
}

```



```

p.push_back({cx,cy+10000*s});
vector<T>tri= {
    {
        n,n+1,n+2
    }
};
;
for(int x=0;x<n;x++) {
    map<pair<int,int>,pair<int,pair<int,int>>>>edges;
    vector<char>bad(tri.size());
    for(int i=0;i<(int)tri.size();i++)if(incircle(p[tri[i].a],p[tri[i].b],p[tri[i].c],p[x])) {
        bad[i]=1;
        for(auto e:{pair{tri[i].a,tri[i].b},pair{tri[i].b,tri[i].c},pair{tri[i].c,tri[i].a}}) {
            auto key=minmax(e.first,e.second);
            edges[key].first++;
            edges[key].second=e;
        }
    }
    vector<T>next;
    for(int i=0;i<(int)tri.size();i++)if(!bad[i])next.push_back(tri[i]);
    for(auto[key,data]:edges)if(data.first==1) {
        auto[u,v]=data.second;
        if(orient(p[u],p[v],p[x])<0)swap(u,v);
        next.push_back({u,v,x});
    }
    tri.swap(next);
}
vector<T>clean;
set<pair<int,int>>>edges;
for(T t:tri)if(max({t.a,t.b,t.c})<n) {
    clean.push_back(t);
    edges.insert(minmax(t.a,t.b));
    edges.insert(minmax(t.b,t.c));
    edges.insert(minmax(t.c,t.a));
}
return {
    clean,vector<pair<int,int>>>(edges.begin(),edges.end())
};
;
}

double shortest_route(const vector<P>& p,int source,int sink) {
    auto[tri,edges]=delaunay(p);
    vector<vector<pair<int,double>>>> graph(p.size());
    for(auto[u,v]:edges) {
        double weight=norm(p[u]-p[v]);
        graph[u].push_back({v,weight});
        graph[v].push_back({u,weight});
    }
    vector<double>distance(p.size(),numeric_limits<double>::infinity());
    priority_queue<pair<double,int>,vector<pair<double,int>>>,
        greater<pair<double,int>>>queue;
    distance[source]=0;
    queue.push({0,source});
    while(!queue.empty()) {
        auto[current,u]=queue.top();queue.pop();
        if(current!=distance[u])continue;
        for(auto[v,weight]:graph[u])if(current+weight<distance[v]) {
            distance[v]=current+weight;
            queue.push({distance[v],v});
        }
    }
    return distance[sink];
}
}

```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    int n,source,sink;
    cin>>n>>source>>sink;
    --source;--sink;
    vector<P>p(n);
    for(auto&x:p)cin>>x.x>>x.y;
    cout<<shortest_route(p,source,sink)<<'\n';
}

```

Python

```

import sys, math, random, heapq
from collections import deque
EPS = 1e-10

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def norm(a):
    return math.hypot(a[0], a[1])

def circle3(a, b, c):
    d = 2 * cross(sub(b, a), sub(c, a))
    ux = (dot(a, a) * (b[1] - c[1]) + dot(b, b) * (c[1] - a[1]) + dot(c, c) * (a[1] - b[1])) / d
    uy = (dot(a, a) * (c[0] - b[0]) + dot(b, b) * (a[0] - c[0]) + dot(c, c) * (b[0] - a[0])) / d
    center = (ux, uy)
    return (center, norm(sub(center, a)))

def orient(a, b, c):
    return cross(sub(b, a), sub(c, a))

def in_circumcircle(a, b, c, p):
    ax = a[0] - p[0]
    ay = a[1] - p[1]
    bx = b[0] - p[0]
    by = b[1] - p[1]
    cx = c[0] - p[0]
    cy = c[1] - p[1]
    det = (ax * ax + ay * ay) * (bx * cy - by * cx) - (bx * bx + by * by) * (ax * cy - ay * cx) + (cx * cx + cy * cy) * (ax * by - ay * bx)
    return det > EPS

def delaunay(points):
    n = len(points)
    if n < 3:
        return ([], [])
    minx = min((x for x, y in points))
    maxx = max((x for x, y in points))
    miny = min((y for x, y in points))
    maxy = max((y for x, y in points))
    span = max(maxx - minx, maxy - miny, 1.0)
    cx = (minx + maxx) / 2
    cy = (miny + maxy) / 2
    p = points + [(cx - 10000 * span, cy - 8000 * span), (cx + 10000 * span, cy - 8000 * span), (cx, cy + 10000 * span)]

```

```

tri = [(n, n + 1, n + 2)]
for idx in range(n):
    bad = [t for t in tri if in_circumcircle(p[t[0]], p[t[1]], p[t[2]], p[idx])]
    count = {}
    direct = {}
    for t in bad:
        for e in ((t[0], t[1]), (t[1], t[2]), (t[2], t[0])):
            key = tuple(sorted(e))
            count[key] = count.get(key, 0) + 1
            direct[key] = e
    gone = set(bad)
    tri = [t for t in tri if t not in gone]
    for key, cnt in count.items():
        if cnt == 1:
            u, v = direct[key]
            if orient(p[u], p[v], p[idx]) < 0:
                u, v = (v, u)
            tri.append((u, v, idx))
tri = [t for t in tri if max(t) < n]
edges = set()
for a, b, c in tri:
    edges |= {tuple(sorted((a, b))), tuple(sorted((b, c))), tuple(sorted((c, a)))}
return (tri, sorted(edges))

def shortest_route(points, source, sink):
    _, edges = delaunay(points)
    graph = [[] for _ in points]
    for u, v in edges:
        weight = math.dist(points[u], points[v])
        graph[u].append((v, weight))
        graph[v].append((u, weight))
    distance = [math.inf] * len(points)
    distance[source] = 0.0
    queue = [(0.0, source)]
    while queue:
        current, u = heapq.heappop(queue)
        if current != distance[u]:
            continue
        for v, weight in graph[u]:
            candidate = current + weight
            if candidate < distance[v]:
                distance[v] = candidate
                heapq.heappush(queue, (candidate, v))
    return distance[sink]

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n, source, sink = map(int, d[:3])
    p = [tuple(d[3 + 2 * i:5 + 2 * i]) for i in range(n)]
    print(f'{shortest_route(p, source - 1, sink - 1):.10f}')
if __name__ == '__main__':
    main()

```

F. Low-Cost Fiber

How to find it

Build Delaunay edges and run Kruskal on that sparse graph.

Correctness

Every Euclidean MST edge has an empty-disk certificate and belongs to Delaunay, so discarding all other complete-graph edges preserves an optimum.

Complexity

Expected $O(n^2 + n \log n)$ here.

C++

```
#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        }
    };
}
P operator-(P b)const {
    return {
        x-b.x,y-b.y
    };
}
P operator*(double k)const {
    return {
        x*k,y*k
    };
}
}
;
double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
}
struct L {
    P p,v;
    double angle;
}
;
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
struct C {
    P p;
    double r;
}
;
double overlap(C a,C b) {
    double d=norm(a.p-b.p);
```

```

    if(d>=a.r+b.r)return 0;
    if(d<=abs(a.r-b.r))return PI*min(a.r,b.r)*min(a.r,b.r);
    double x=acos(clamp((d*d+a.r*a.r-b.r*b.r)/(2*d*a.r),-1.0,1.0)),y=acos(clamp((d*d+b.r*b.r-a.r*a.r)/(2*d*b.r),-1.0,1.0));
    return a.r*a.r*x+b.r*b.r*y-d*a.r*sin(x);
}
int tangents(C a,C b) {
    double d=norm(a.p-b.p);
    if(d<=EPS)return abs(a.r-b.r)<=EPS?-1:0;
    int ans=0;
    for(double z:{abs(a.r-b.r),a.r+b.r})ans+=d>z+EPS?2:(abs(d-z)<=EPS);
    return ans;
}
C circle2(P a,P b) {
    P c=(a+b)*.5;
    return {
        c,norm(a-b)/2
    };
}
C circle3(P a,P b,P c) {
    double d=2*cross(b-a,c-a);
    double aa=dot(a,a),bb=dot(b,b),cc=dot(c,c);
    P o= {
        (aa*(b.y-c.y)+bb*(c.y-a.y)+cc*(a.y-b.y))/d,(aa*(c.x-b.x)+bb*(a.x-c.x)+cc*(b.x-a.x))/d
    };
    return {
        o,norm(o-a)
    };
}
C mec(vector<P>p) {
    mt19937 rng(947311);
    shuffle(p.begin(),p.end(),rng);
    C c {
        {
            0,0
        },-1
    };
    auto outside=[](P x,C c) {
        return c.r<0||norm(x-c.p)>c.r+1e-9;
    };
    for(int i=0;i<(int)p.size();i++)if(outside(p[i],c)) {
        c= {
            p[i],0
        };
        for(int j=0;j<i;j++)if(outside(p[j],c)) {
            c=circle2(p[i],p[j]);
            for(int k=0;k<j;k++)if(outside(p[k],c))c=circle3(p[i],p[j],p[k]);
        }
    }
    return c;
}
struct T {
    int a,b,c;
};
double orient(P a,P b,P c) {
    return cross(b-a,c-a);
}

```

```

}
bool incircle(P a,P b,P c,P p) {
    double ax=a.x-p.x,ay=a.y-p.y,bx=b.x-p.x,by=b.y-p.y,cx=c.x-p.x,cy=c.y-p.y;
    double d=(ax*ax+ay*ay)*(bx*cy-by*cx)-(bx*bx+by*by)*(ax*cy-ay*cx)+(cx*cx+cy*cy)*(ax*by-ay*bx);
    return d>EPS;
}
pair<vector<T>,vector<pair<int,int>>>>deLaunay(vector<P>p) {
    int n=p.size();
    if(n<3)return {
        {
        },
        {
        }
    }
    ;
    double minx=p[0].x,maxx=p[0].x,miny=p[0].y,maxy=p[0].y;
    for(P x:p)minx=min(minx,x.x),maxx=max(maxx,x.x),miny=min(miny,x.y),maxy=max(maxy,x.y);
    double s=max({maxx-minx,maxy-miny,1.0}),cx=(minx+maxx)/2,cy=(miny+maxy)/2;
    p.push_back({cx-10000*s,cy-8000*s});
    p.push_back({cx+10000*s,cy-8000*s});
    p.push_back({cx,cy+10000*s});
    vector<T>tri= {
        {
            n,n+1,n+2
        }
    }
    ;
    for(int x=0;x<n;x++) {
        map<pair<int,int>,pair<int,pair<int,int>>>>edges;
        vector<char>bad(tri.size());
        for(int i=0;i<(int)tri.size();i++)if(incircle(p[tri[i].a],p[tri[i].b],p[tri[i].c],p[x])) {
            bad[i]=1;
            for(auto e:{pair{tri[i].a,tri[i].b},pair{tri[i].b,tri[i].c},pair{tri[i].c,tri[i].a}}) {
                auto key=minmax(e.first,e.second);
                edges[key].first++;
                edges[key].second=e;
            }
        }
        vector<T>next;
        for(int i=0;i<(int)tri.size();i++)if(!bad[i])next.push_back(tri[i]);
        for(auto [key,data]:edges)if(data.first==1) {
            auto [u,v]=data.second;
            if(orient(p[u],p[v],p[x])<0)swap(u,v);
            next.push_back({u,v,x});
        }
        tri.swap(next);
    }
    vector<T>clean;
    set<pair<int,int>>>edges;
    for(T t:tri)if(max({t.a,t.b,t.c})<n) {
        clean.push_back(t);
        edges.insert(minmax(t.a,t.b));
        edges.insert(minmax(t.b,t.c));
        edges.insert(minmax(t.c,t.a));
    }
    return {
        clean,vector<pair<int,int>>>(edges.begin(),edges.end())
    }
    ;
}
double radii(vector<P>p,bool maximum) {
    auto [tri,e]=deLaunay(p);
    double ans=0;

```

```

    for(T t:tri) {
        double r=circle3(p[t.a],p[t.b],p[t.c]).r;
        if(maximum)ans=max(ans,r);
        else ans+=r;
    }
    return ans;
}
}

struct DSU {
    vector<int>p;
    DSU(int n):p(n) {
        iota(p.begin(),p.end(),0);
    }
    int f(int x) {
        return p[x]==x?x:p[x]=f(p[x]);
    }
}

;
double mst(vector<P>p) {
    int n=p.size();
    if(n<2)return 0;
    auto[tri,e]=delaunay(p);
    if(e.empty())for(int i=1;i<n;i++)e.push_back({i-1,i});
    vector<tuple<double,int,int>>all;
    for(auto[u,v]:e)all.push_back({norm(p[u]-p[v]),u,v});
    sort(all.begin(),all.end());
    DSU d(n);
    double ans=0;
    for(auto[w,u,v]:all) {
        u=d.f(u);
        v=d.f(v);
        if(u!=v)d.p[u]=v,ans+=w;
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    int n;
    cin>>n;
    vector<P>p(n);
    for(auto&x:p)cin>>x.x>>x.y;
    cout<<mst(p)<<'\n';
}

```

Python

```

import sys, math, random
from collections import deque
EPS = 1e-10

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def norm(a):
    return math.hypot(a[0], a[1])

def orient(a, b, c):
    return cross(sub(b, a), sub(c, a))

```

```

def in_circumcircle(a, b, c, p):
    ax = a[0] - p[0]
    ay = a[1] - p[1]
    bx = b[0] - p[0]
    by = b[1] - p[1]
    cx = c[0] - p[0]
    cy = c[1] - p[1]
    det = (ax * ax + ay * ay) * (bx * cy - by * cx) - (bx * bx + by * by) * (ax * cy - ay * cx) + (cx * cx + cy * cy) *
    return det > EPS

def delaunay(points):
    n = len(points)
    if n < 3:
        return ([], [])
    minx = min((x for x, y in points))
    maxx = max((x for x, y in points))
    miny = min((y for x, y in points))
    maxy = max((y for x, y in points))
    span = max(maxx - minx, maxy - miny, 1.0)
    cx = (minx + maxx) / 2
    cy = (miny + maxy) / 2
    p = points + [(cx - 10000 * span, cy - 8000 * span), (cx + 10000 * span, cy - 8000 * span), (cx, cy + 10000 * span)]
    tri = [(n, n + 1, n + 2)]
    for idx in range(n):
        bad = [t for t in tri if in_circumcircle(p[t[0]], p[t[1]], p[t[2]], p[idx])]
        count = {}
        direct = {}
        for t in bad:
            for e in ((t[0], t[1]), (t[1], t[2]), (t[2], t[0])):
                key = tuple(sorted(e))
                count[key] = count.get(key, 0) + 1
                direct[key] = e
        gone = set(bad)
        tri = [t for t in tri if t not in gone]
        for key, cnt in count.items():
            if cnt == 1:
                u, v = direct[key]
                if orient(p[u], p[v], p[idx]) < 0:
                    u, v = (v, u)
                tri.append((u, v, idx))
    tri = [t for t in tri if max(t) < n]
    edges = set()
    for a, b, c in tri:
        edges |= {tuple(sorted((a, b))), tuple(sorted((b, c))), tuple(sorted((c, a)))}
    return (tri, sorted(edges))

def mst(points):
    n = len(points)
    if n < 2:
        return 0.0
    _, edges = delaunay(points)
    if not edges:
        edges = [(i, i + 1) for i in range(n - 1)]
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    ans = 0.0
    for w, u, v in sorted(((norm(sub(points[u], points[v])), u, v) for u, v in edges)):
        u = find(u)

```



```

        v = find(v)
        if u != v:
            parent[u] = v
            ans += w
    return ans

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n = int(d[0])
    p = [tuple(d[1 + 2 * i:3 + 2 * i]) for i in range(n)]
    print(f'{mst(p):.10f}')
if __name__ == '__main__':
    main()

```